LAB 7

Fuzzing for Input Validation Bugs

Student: C DINESH KUMAR
SRN: PES1UG24CS117
Application: Car Insurance Premium Parser

Lab submission report

This report explains the input-validation problems found through Hypothesis fuzz testing, the defensive fixes applied to the three functions, and the execution evidence.

Deliverables included: fixed Python source, completed Hypothesis tests, and four screenshots showing the original run, failing tests, fixed run, and passing tests.

1. Objective and problem identified

The objective of this lab was to use fuzz testing with the Hypothesis library to discover input-validation bugs. Instead of checking only expected inputs, the tests supply generated text values and None to see whether each function can handle unexpected input without crashing.

What was wrong in the original application?

Function	Problem found	Example failure
normalize_registration()	It called string methods directly without checking the input type.	None caused an AttributeError because None has no strip() method.
total_claim_amount()	It attempted to convert every semicolon-separated part to an integer without validating empty or invalid entries, and assumed the input was a string.	Empty text or an empty part could cause ValueError; None could cause AttributeError.
driver_age()	It assumed the date string contained a third slash-separated part and a valid year.	Empty or malformed input could cause IndexError or ValueError; None could cause AttributeError.

The important observation is that the original program could work with normal examples while still failing on edge cases. The failing Hypothesis run (SS2) shows that the generated inputs exposed these weaknesses.

2. Fixes applied to the application

A. normalize_registration(plate)

Added an input-type check. If plate is not a string, the function returns an empty string. For string input, it trims leading/trailing whitespace, converts the value to uppercase, and removes spaces, hyphens, and dots. This prevents non-string inputs such as None from causing a crash.

Example: ka 01 ab 1234 → KA01AB1234

B. total_claim_amount(text)

Added an input-type check. Non-string input returns 0. The function now processes each semicolon-separated entry individually, removes surrounding whitespace, and adds the entry only when it contains digits. Empty or invalid entries are ignored instead of being passed directly to int().

Example: 12000;4500;800 → 17300

C. driver_age(dob, current_year=2026)

Added an input-type check and used datetime.strptime() to validate the complete date against the DD/MM/YYYY format. A future birth year or invalid date returns None. ValueError is handled, so malformed dates do not terminate the program. For a valid date, the function returns current_year minus the birth year.

Example: 14/08/1999 → 27 (using current_year=2026)

Overall improvement

The changes use defensive programming: validate the input type, validate the content before converting or processing it, and return a safe result for invalid input. The normal sample inputs continue to produce the expected outputs.

3. Hypothesis tests and results

The test file contains one property-based test for each function. Each test uses the Hypothesis strategy **st.text() | st.none()**, which generates strings (including unusual or empty strings) as well as None. The test calls the function inside try/except and fails the test if an exception is raised.

Test cases

- **test_normalize_registration_no_crash:** checks that normalize_registration() does not raise an exception for generated text or None.
- **test_total_claim_amount_no_crash:** checks that total_claim_amount() does not raise an exception for generated text or None.
- **test_driver_age_no_crash:** checks that driver_age() does not raise an exception for generated text or None.

Execution sequence

1. Run the original application with sample inputs (SS1).
2. Run the Hypothesis tests against the original functions and observe failures (SS2).
3. Add input validation and defensive handling to the three functions, then run the fixed application (SS3).
4. Run the same tests again and confirm that all three pass (SS4).

Observed result

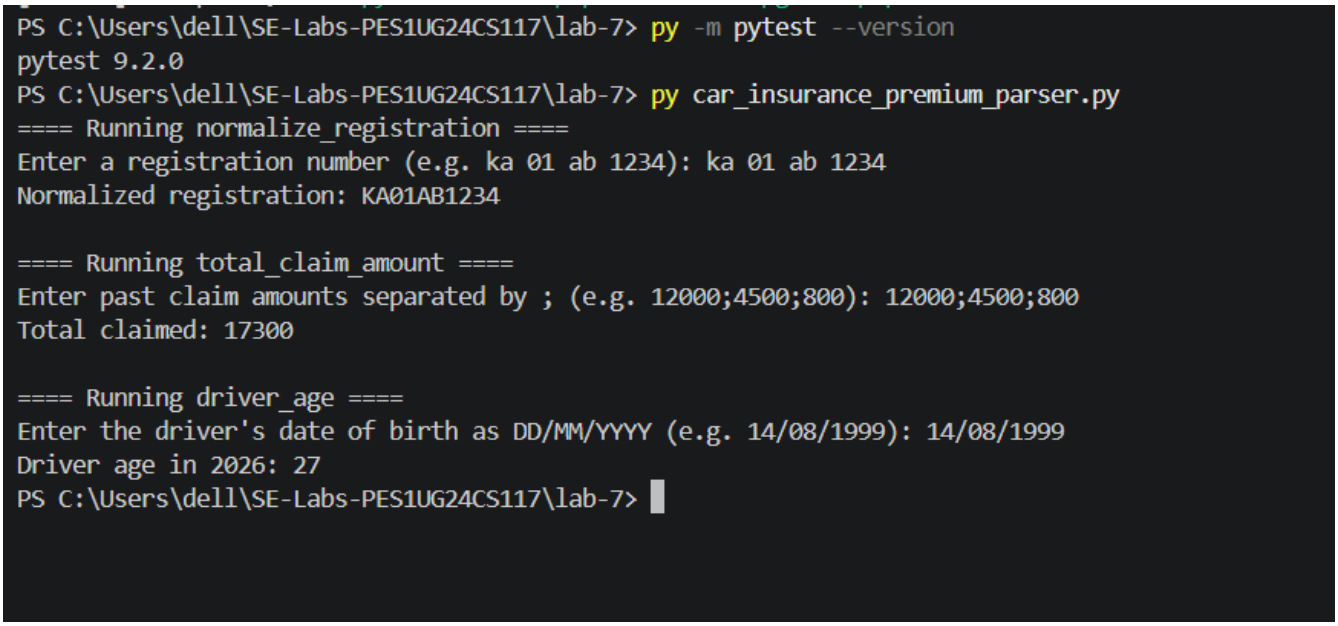
Before fixing: all three tests failed because Hypothesis found inputs that triggered exceptions.
After fixing: all three tests passed in the displayed pytest run (3 passed).

Scope note: These tests establish the no-crash property for the generated text and None inputs. They do not independently prove every possible business-rule result is correct; separate assertions would be needed to test exact outputs for a wider set of valid and invalid cases.

4. Screenshot evidence

SS1 – Original application

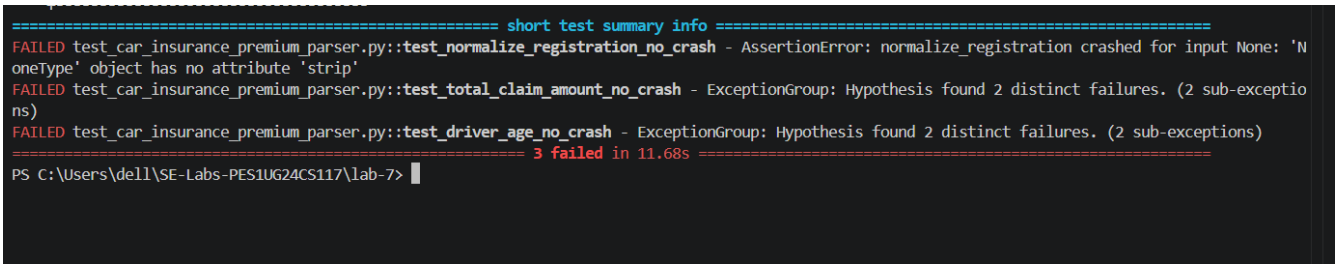
The original application is run with normal sample inputs. These normal inputs work, but this does not prove that the functions can handle unexpected or malformed inputs.



Screenshot evidence

SS2 – Hypothesis tests expose failures

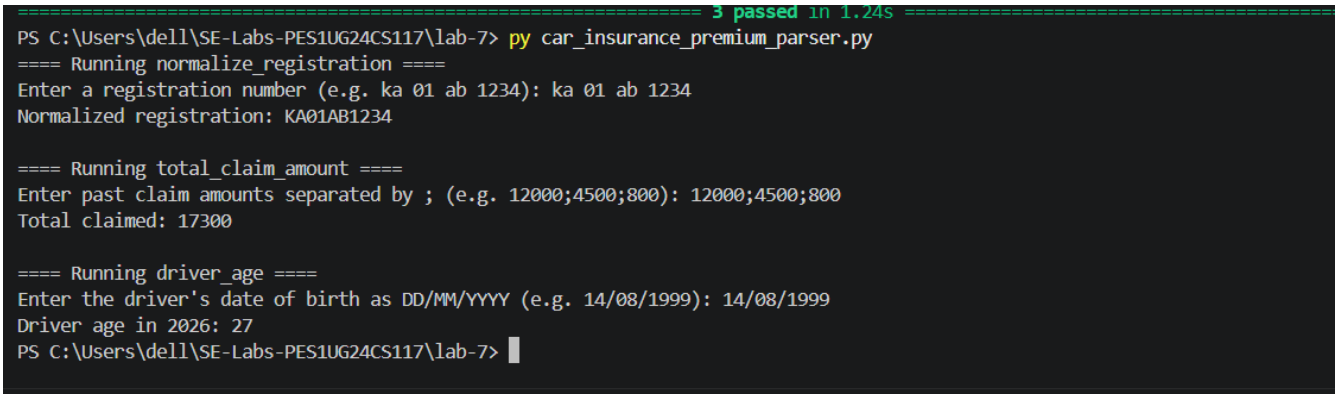
The tests generate unexpected strings and None values. The original functions raise exceptions for some generated inputs, demonstrating missing input validation.



Screenshot evidence

SS3 – Fixed application

The updated application is run with valid sample inputs to confirm that the expected outputs are still produced.



Screenshot evidence

SS4 – Hypothesis tests pass

After the fixes, all three no-crash tests pass in the displayed pytest run.

